Movie Theater Ticketing System (MTTS)

Software Design Specification

Course Project Deliverable 2 — Section 4 of the MTTS SRS

Benjamin Nguyen	*Author (solo project)*
Gus Hanna, Ph.D.	*Instructor, CS 250*

July 20, 2026

4. Software Design Specification

This section is the second deliverable of the course project and extends the Movie Theater Ticketing System (MTTS) SRS with the software design. Where Sections 1–3 were written for the client, this section is written for the developers who will implement and later maintain the system. It describes the software architecture and implementation at the level of a UML class diagram: the classes and data structures, their attributes and datatypes, and their major operations with full parameter lists, followed by the development plan and timeline.

4.1 System Description

MTTS is a browser-based ticketing system for a San Diego–area chain of 20 theaters, replacing the client's slow and defect-prone legacy system. Any user can browse the movies playing at each theater, the 6–8 daily showtimes, ticket prices, live seat/ticket availability, and third-party review scores with critic quotes. A customer purchases up to 20 tickets per transaction, selects assigned seats when the showtime is in a deluxe auditorium, pays by credit card or PayPal through an external processor, and receives uniquely coded tickets digitally by email or printed at an in-theater kiosk. Customer accounts and memberships are optional, and student, military, and senior discounts apply on weekdays.

Theater staff use an administrator mode to create, edit, and cancel showtimes and to override erroneous purchases, which is also how in-person refunds are supported (in-system refunds are out of scope). Management can report tickets sold and ticket prices per movie. The website and in-theater kiosks are served by the same back end, so both always present the same live availability data, and the system is designed to support at least 1,000 concurrent users.

4.2 Software Architecture Overview

MTTS uses a three-tier architecture. Thin clients (customer browsers, kiosks, and the administrator console) talk over HTTPS to a single C++ application server, which is the only component that reads or writes the central database. Placing all business logic in one shared server tier is what guarantees the SRS requirement that online users and kiosks see consistent, live-updating availability, and it keeps the external payment, review, and email services isolated behind adapter modules.

4.2.1 Architectural Diagram

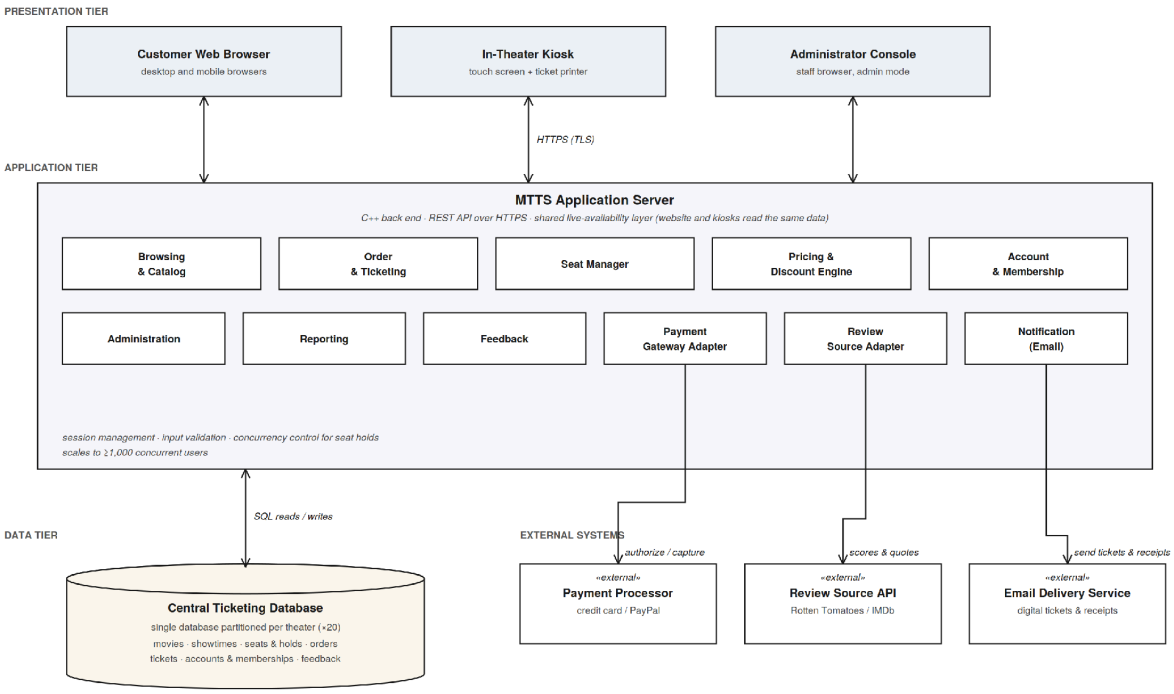

Figure 4.1 — MTTS three-tier architecture with external systems.

The responsibilities of the major components are:

Component	Responsibility
Customer Web Browser	Customer-facing UI for desktop and mobile browsers: browsing, seat selection, checkout, account pages, and feedback.
In-Theater Kiosk	Touch-screen front end running the same flows against the same API; prints physical tickets on its attached ticket printer.
Administrator Console	Browser UI for staff: showtime management, purchase override, and reports.
Browsing & Catalog	Serves movie/showtime listings with prices, availability, auditorium type, and cached review data.
Order & Ticketing	Creates orders, enforces the 20-ticket limit, generates unique ticket codes, and finalizes checkout.
Seat Manager	Places, expires, and releases per-seat holds for deluxe showtimes under concurrent access.
Pricing & Discount Engine	Computes ticket prices and validates weekday-only student/military/senior discounts.
Account & Membership	Registration, login (salted password hashes), order history, and membership points.
Administration	Showtime create/edit/cancel and purchase override for staff.

Component	Responsibility
Reporting	Aggregates tickets sold and ticket prices per movie for management.
Feedback	Accepts and stores post-purchase ratings and comments.
Notification (Email)	Sends digital tickets and receipts through the external email delivery service.
Payment Gateway Adapter	Wraps the external credit-card/PayPal processor; no card data is stored in MTTS.
Review Source Adapter	Fetches review scores and critic quotes from the third-party review API on a daily schedule.
Central Ticketing Database	Single database partitioned per theater (×20) holding movies, showtimes, seats and holds, orders, tickets, accounts, and feedback; all seat-hold and checkout mutations are transactional.

The connectors joining these components are:

Connector	Description
Clients ↔ Application Server	HTTPS/TLS REST calls. The web browser, kiosk, and administrator console all use the same API, so seat availability updates live across every channel; requests carry session tokens and are validated server-side.
Application Server ↔ Central Ticketing Database	Transactional SQL reads and writes routed to the correct per-theater partition; seat holds and checkout commits execute as atomic transactions.
Payment Gateway Adapter ↔ Payment Processor	Outbound HTTPS authorize/capture calls to the external credit-card/PayPal processor; only an authorization token is returned and stored.
Review Source Adapter ↔ Review Source API	Outbound HTTPS fetch of review scores and critic quotes on a daily schedule; results are cached in the database.
Notification (Email) ↔ Email Delivery Service	Outbound HTTPS send of digital tickets and purchase receipts to the customer's email address.

4.2.2 UML Class Diagram

Figure 4.2 (next page) shows the class model. Notation: "−" marks private attributes and "+" public operations; filled diamonds are compositions; dashed arrows are dependencies; classes marked «service» are the stateless controller and adapter classes of the application server. Types are C++/STL types (an attribute title : string is implemented as std::string title). Multiplicities encode the business rules from the SRS — for example, an Order owns 1..20 Tickets and is linked to 0..1 Customer because guests may purchase without an account.

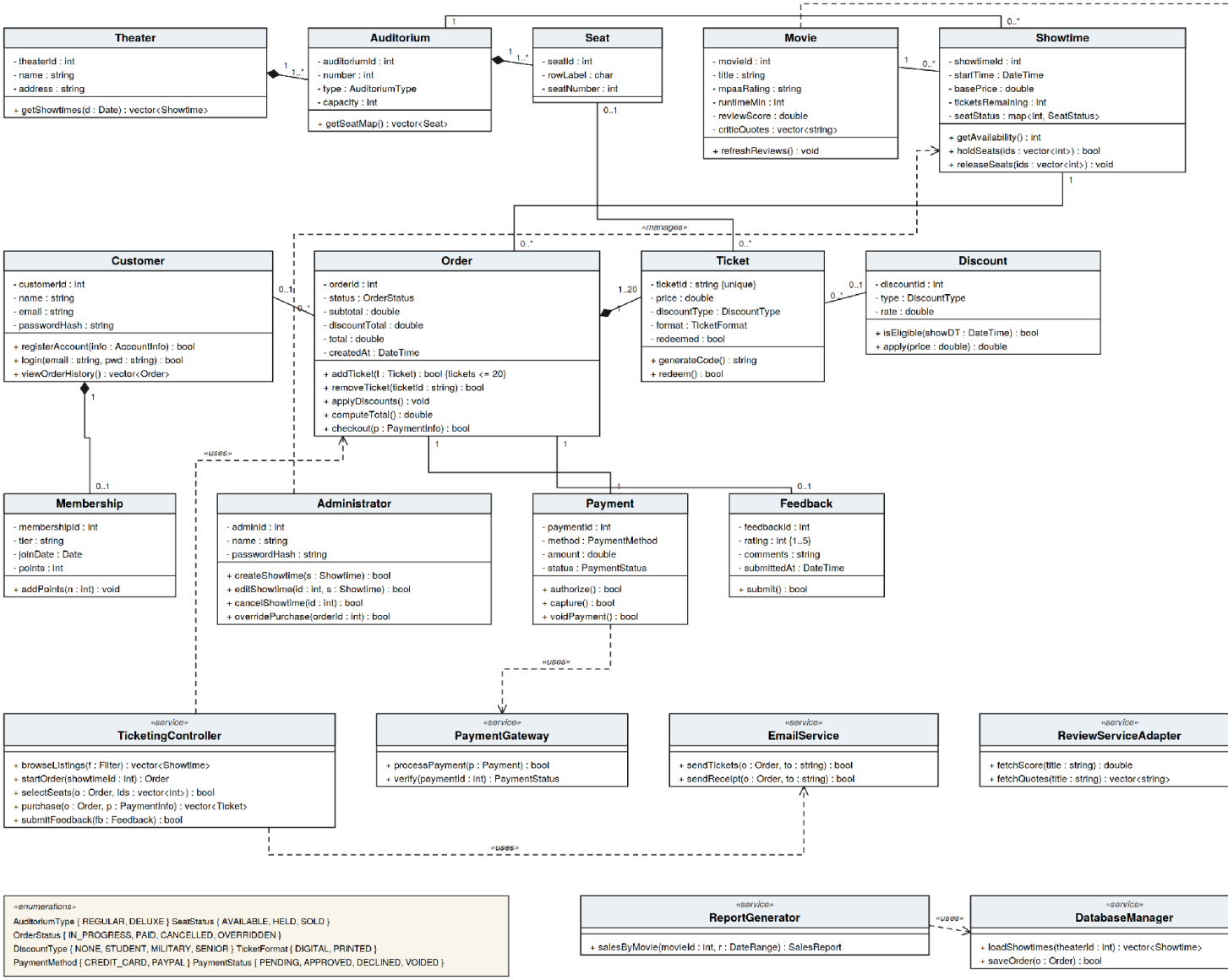

Figure 4.2 — MTTS UML class diagram.

4.2.3 Class Descriptions

Each class below is described with its purpose, its attributes (name, datatype, meaning), and its operations with full parameter lists and return types. Operations shown in the diagram and the tables are identical.

4.2.3.1 Movie

Represents a film that the chain is showing. Review data is cached from the third-party review source so listings can render without a live external call on every request (SRS 3.1.3.3).

Attribute	Type	Description
`movieId`	`int`	Unique identifier (database primary key).
`title`	`string`	Movie title displayed in listings.
`mpaaRating`	`string`	MPAA rating (G, PG, PG-13, R, NC-17).
`runtimeMin`	`int`	Running time in minutes.
`reviewScore`	`double`	Cached aggregate critic score (0–100) from the review source.
`criticQuotes`	`vector<string>`	Cached critic quotes shown with the listing.

Operation	Description
`refreshReviews() : void`	Calls ReviewServiceAdapter to update reviewScore and criticQuotes; run on a daily schedule.

4.2.3.2 Theater

One physical theater location in the 20-theater chain. Each theater maps to one database partition (SRS 3.1.3.1).

Attribute	Type	Description
`theaterId`	`int`	Unique identifier; also selects the database partition.
`name`	`string`	Display name of the location.
`address`	`string`	Street address shown to customers.

Operation	Description
`getShowtimes(d : Date) : vector<Showtime>`	Returns all showtimes (6–8 per day) scheduled at this theater on date d.

4.2.3.3 Auditorium

A single screen inside a theater. The type attribute distinguishes regular auditoriums (general admission) from deluxe auditoriums, which require assigned seating.

Attribute	Type	Description
auditoriumId	int	Unique identifier.
number	int	Auditorium number within the theater.
type	AuditoriumType	REGULAR or DELUXE; controls whether seat selection is required.
capacity	int	Total number of seats; upper bound on tickets sold per showtime.

Operation	Description
getSeatMap() : vector<Seat>	Returns the physical seat layout, used by the front end to draw the seat-selection map for deluxe showtimes.

4.2.3.4 Seat

A physical seat within an auditorium. Seat availability is not stored here — it is per showtime, so it lives in Showtime.seatStatus.

Attribute	Type	Description
seatId	int	Unique identifier.
rowLabel	char	Row letter (A, B, C, …).
seatNumber	int	Seat number within the row.

Operations: none — plain data class accessed through Auditorium and Showtime.

4.2.3.5 Showtime

A scheduled screening of one Movie in one Auditorium. It is the concurrency-critical class: seat holds and ticket counts must stay consistent for up to 1,000 concurrent users across the website and kiosks, so all mutations are executed inside database transactions.

Attribute	Type	Description
showtimeId	int	Unique identifier.
startTime	DateTime	Date and time the screening begins.
basePrice	double	Undiscounted ticket price for this showtime.
ticketsRemaining	int	Tickets still available (regular auditoriums).
seatStatus	map<int, SeatStatus>	Per-seat state (AVAILABLE, HELD, SOLD) keyed by seatId; used for deluxe auditoriums.

Operation	Description
getAvailability() : int	Returns remaining tickets/seats; the same live value is served to the website and kiosks.

Operation	Description
`holdSeats(ids : vector<int>) : bool`	Atomically places a short-lived hold on the given seats during checkout; returns false if any seat is no longer available.
`releaseSeats(ids : vector<int>) : void`	Releases holds when a checkout is abandoned, times out, or fails.

4.2.3.6 Customer

An optional registered account (SRS: accounts and memberships are optional — guests may purchase without one). Passwords are stored only as salted hashes.

Attribute	Type	Description
`customerId`	`int`	Unique identifier.
`name`	`string`	Customer's display name.
`email`	`string`	Login identifier; also the delivery address for digital tickets and receipts.
`passwordHash`	`string`	Salted hash of the password; the plain password is never stored.

Operation	Description
`registerAccount(info : AccountInfo) : bool`	Creates the account after validating the email is unused; returns success.
`login(email : string, pwd : string) : bool`	Verifies credentials against the stored hash and opens a session.
`viewOrderHistory() : vector<Order>`	Returns this customer's past orders.

4.2.3.7 Membership

Optional loyalty membership attached to a Customer account; owned by the Customer (composition).

Attribute	Type	Description
`membershipId`	`int`	Unique identifier.
`tier`	`string`	Membership tier name.
`joinDate`	`Date`	Date the membership started.
`points`	`int`	Accumulated loyalty points.

Operation	Description
`addPoints(n : int) : void`	Credits points after a completed purchase.

4.2.3.8 Administrator

A staff account with access to administrator mode. Supports the two admin features in the SRS: managing showtimes and overriding erroneous purchases (which is how in-person refunds are supported, since in-system refunds are out of scope).

Attribute	Type	Description
`adminId`	`int`	Unique identifier.
`name`	`string`	Staff member name.
`passwordHash`	`string`	Salted hash of the admin password.

Operation	Description
`createShowtime(s : Showtime) : bool`	Adds a new showtime to the schedule.
`editShowtime(id : int, s : Showtime) : bool`	Updates time, price, or auditorium of an existing showtime.
`cancelShowtime(id : int) : bool`	Removes a showtime; fails if tickets are already sold unless purchases are overridden first.
`overridePurchase(orderId : int) : bool`	Marks an order OVERRIDDEN, invalidates its tickets, and returns the seats/tickets to availability; performed by an employee for erroneous purchases and in-person refunds.

4.2.3.9 Order

One purchase transaction. It owns its Tickets (composition) and enforces the business rule of at most 20 tickets per transaction. The customer link is optional so that guests can buy without an account.

Attribute	Type	Description
`orderId`	`int`	Unique identifier.
`status`	`OrderStatus`	IN_PROGRESS, PAID, CANCELLED, or OVERRIDDEN.
`subtotal`	`double`	Sum of undiscounted ticket prices.
`discountTotal`	`double`	Total amount subtracted by discounts.
`total`	`double`	Final amount charged: subtotal − discountTotal.
`createdAt`	`DateTime`	When the order was started; used to expire abandoned seat holds.

Operation	Description
`addTicket(t : Ticket) : bool`	Adds a ticket to the order; returns false if the order already holds 20 tickets (SRS limit).
`removeTicket(ticketId : string) : bool`	Removes a ticket before checkout and releases its seat hold.
`applyDiscounts() : void`	Asks Discount.isEligible() for each ticket's requested discount and updates each ticket's price and discountTotal.

Operation	Description
`computeTotal() : double`	Recomputes subtotal, discountTotal, and total from the current ticket list.
`checkout(pay : PaymentInfo) : bool`	Creates a Payment, runs it through PaymentGateway, and on approval marks the order PAID, finalizes seats as SOLD, generates ticket codes, and triggers EmailService delivery.

4.2.3.10 Ticket

One admission ticket inside an Order. Each ticket carries a unique code so it can be issued digitally or printed at a kiosk and validated exactly once at the door. The seat link is present only for deluxe showtimes.

Attribute	Type	Description
`ticketId`	`string`	Unique, hard-to-guess code (UUID) printed/encoded on the ticket.
`price`	`double`	Price actually paid for this ticket after any discount.
`discountType`	`DiscountType`	NONE, STUDENT, MILITARY, or SENIOR.
`format`	`TicketFormat`	DIGITAL (emailed) or PRINTED (kiosk ticket printer).
`redeemed`	`bool`	Set true when the ticket is scanned at the theater; prevents reuse.

Operation	Description
`generateCode() : string`	Generates and stores the unique ticketId at checkout.
`redeem() : bool`	Atomically marks the ticket used; returns false if already redeemed or its order was overridden.

4.2.3.11 Discount

A discount rule. Per the SRS, student, military, and senior discounts apply on weekdays only, which isEligible() enforces from the showtime's date.

Attribute	Type	Description
`discountId`	`int`	Unique identifier.
`type`	`DiscountType`	STUDENT, MILITARY, or SENIOR.
`rate`	`double`	Fractional reduction, e.g. 0.20 for 20% off.

Operation	Description
`isEligible(showDT : DateTime) : bool`	True only when the showtime falls on a weekday (Mon–Fri).
`apply(price : double) : double`	Returns price × (1 − rate), rounded to cents.

4.2.3.12 Payment

The record of one payment attempt for an Order, processed externally through PaymentGateway. Card and PayPal data are handed to the external processor and never stored by MTTS.

Attribute	Type	Description
paymentId	int	Unique identifier.
method	PaymentMethod	CREDIT_CARD or PAYPAL.
amount	double	Amount charged; equals Order.total.
status	PaymentStatus	PENDING, APPROVED, DECLINED, or VOIDED.

Operation	Description
authorize() : bool	Requests authorization from the external processor via PaymentGateway.
capture() : bool	Captures an authorized payment; sets status APPROVED.
voidPayment() : bool	Voids the payment if checkout fails after authorization.

4.2.3.13 Feedback

Optional post-purchase feedback tied to one Order, satisfying the SRS feedback feature.

Attribute	Type	Description
feedbackId	int	Unique identifier.
rating	int	Star rating, constrained to 1–5.
comments	string	Free-text comments.
submittedAt	DateTime	Submission timestamp.

Operation	Description
submit() : bool	Validates the rating range and persists the feedback.

4.2.3.14 TicketingController «service»

The application-server facade that the web and kiosk front ends call over the REST API. It orchestrates the domain classes for the main use cases; keeping both front ends on this single controller is what guarantees the website and kiosks always see the same live data.

Attributes: none — stateless service class.

Operation	Description
`browseListings(f : Filter) : vector<Showtime>`	Returns listings matching optional filters (theater, movie, date, showtime), including prices, availability, auditorium type, and review data.
`startOrder(showtimeId : int) : Order`	Creates an IN_PROGRESS order for the chosen showtime.
`selectSeats(o : Order, ids : vector<int>) : bool`	For deluxe showtimes, places holds via Showtime.holdSeats() and attaches the seats to the order's tickets.
`purchase(o : Order, p : PaymentInfo) : vector<Ticket>`	Runs Order.checkout() and returns the finalized tickets for display, email, or kiosk printing.
`submitFeedback(fb : Feedback) : bool`	Accepts post-purchase feedback for a completed order.

4.2.3.15 PaymentGateway «service»

Adapter that wraps the external payment processor (credit card / PayPal), isolating the rest of the system from the processor's API.

Attributes: none — stateless service class.

Operation	Description
`processPayment(p : Payment) : bool`	Sends the authorization/capture request over HTTPS and updates p.status from the response.
`verify(paymentId : int) : PaymentStatus`	Re-queries the processor for the current status of a payment.

4.2.3.16 ReviewServiceAdapter «service»

Adapter for the third-party review source (e.g., Rotten Tomatoes or IMDb) required by SRS 3.1.3.3.

Attributes: none — stateless service class.

Operation	Description
`fetchScore(title : string) : double`	Retrieves the aggregate critic score for a movie.
`fetchQuotes(title : string) : vector<string>`	Retrieves critic quotes to display with the listing.

4.2.3.17 EmailService «service»

Adapter for the external email delivery service used to send digital tickets and receipts (SRS 3.1.3.4).

Attributes: none — stateless service class.

Operation	Description
`sendTickets(o : Order, to : string) : bool`	Emails the unique ticket codes for a paid order.

Operation	Description
`sendReceipt(o : Order, to : string) : bool`	Emails an itemized receipt including any discounts applied.

4.2.3.18 ReportGenerator «service»

Produces the management reports required by the SRS: tickets sold and ticket prices per movie.

Attributes: none — stateless service class.

Operation	Description
`salesByMovie(movieId : int, r : DateRange) : SalesReport`	Aggregates orders in the date range into a SalesReport struct { ticketsSold : int, revenue : double, avgPrice : double }.

4.2.3.19 DatabaseManager «service»

Data-access layer over the central database. It routes every query to the correct per-theater partition and wraps seat-hold and checkout mutations in transactions so availability stays consistent under concurrent load.

Attributes: none — stateless service class.

Operation	Description
`loadShowtimes(theaterId : int) : vector<Showtime>`	Loads a theater's showtimes (with live availability) from that theater's partition.
`saveOrder(o : Order) : bool`	Persists an order, its tickets, and its payment atomically in one transaction.

4.2.4 Enumerated Types

Enumeration	Values	Used by
`AuditoriumType`	`REGULAR, DELUXE`	`Auditorium.type`
`SeatStatus`	`AVAILABLE, HELD, SOLD`	`Showtime.seatStatus`
`OrderStatus`	`IN_PROGRESS, PAID, CANCELLED, OVERRIDDEN`	`Order.status`
`DiscountType`	`NONE, STUDENT, MILITARY, SENIOR`	`Ticket.discountType, Discount.type`
`TicketFormat`	`DIGITAL, PRINTED`	`Ticket.format`
`PaymentMethod`	`CREDIT_CARD, PAYPAL`	`Payment.method`
`PaymentStatus`	`PENDING, APPROVED, DECLINED, VOIDED`	`Payment.status`

4.3 Development Plan and Timeline

The SRS commits to a user-testable prototype within 4 months. Development is therefore planned as a 16-week schedule in which the database and domain model are built first, the customer-facing purchase path second, and the account, administrative, and reporting features third, leaving the final two weeks for load and security testing against the 1,000-concurrent-user requirement. This is a solo project, so every task below is the responsibility of Benjamin Nguyen and the schedule is sequenced for a single developer with no parallel work streams.

4.3.1 Task List, Timeline, and Responsibilities

Task	Weeks	Work and milestone	Responsible
1. Foundations	1–2	Database schema across per-theater partitions; skeletons of all domain classes; GitHub repository and commit workflow. Milestone: schema review passed.	Benjamin Nguyen
2. Browsing & catalog	3–5	Browsing & Catalog module, ReviewServiceAdapter with daily refresh, listing UI on web and kiosk. Milestone: live listings demo.	Benjamin Nguyen
3. Ordering & seats	6–8	Order/Ticket flow, Seat Manager holds with expiry, deluxe seat-map UI, unique ticket-code generation. Milestone: end-to-end order without payment.	Benjamin Nguyen
4. Payment & delivery	9–10	PaymentGateway integration (card/PayPal), EmailService tickets and receipts, kiosk ticket printing. Milestone: first real purchase in staging.	Benjamin Nguyen
5. Accounts & discounts	11–12	Registration/login, memberships and points, weekday discount validation. Milestone: discounted member purchase.	Benjamin Nguyen
6. Admin, reports, feedback	13–14	Administrator mode (showtimes, purchase override), per-movie sales reports, feedback capture. Milestone: staff walkthrough.	Benjamin Nguyen
7. Hardening & prototype	15–16	Load testing to ≥1,000 concurrent users, security review (HTTPS, hashing, payment isolation), bug fixes, documentation. Milestone: user-testable prototype delivered.	Benjamin Nguyen

4.3.2 Repository and Commit Plan

As the sole team member, Benjamin Nguyen is responsible for all design, implementation, testing, and documentation tasks listed above. All work is committed to the project GitHub repository at the end of each task, satisfying the requirement that every group member push at least one commit for this deliverable.